Client-Side ANN on the Web: A Wikipedia Search Demonstration

David Denisov
University of Haifa
Haifa, Israel

Live search demo: https://wiki-ann-demo-20261022.pages.dev/client_wiki/
Clustering lab: https://wiki-ann-demo-20261022.pages.dev/client_wiki/benchmark.html
Code and benchmarks: https://github.com/DavidDenisov/ANN-on-the-Web

Abstract

We demonstrate browser-based approximate nearest neighbour (ANN) search over 35.2 million English Wikipedia passages hosted on a static CDN. The browser embeds queries, downloads selected vector clusters using inverted file (IVF) routing, ranks their vectors, and fetches text for the top results. Visitors can inspect passages, timings, and download costs for their own questions. A companion lab measures agreement and download size across settings. On a desktop with primed CDN caches, the default settings (32k clusters, SQ6 centroids, four probes) yield an estimated 174 ms for a 128-token query after startup, based on separate means for embedding, vector retrieval, and ten-passage text loading. Worker startup averages 2.46 s. At the same settings, the browser agrees with exact top-1 search on 98.89% of 10,000 queries generated by adding noise of magnitude 0.4 to passage embeddings.

CCS Concepts

• **Information systems → Similarity search**.

Keywords

approximate nearest neighbors, semantic search, client-side retrieval

ACM Reference Format:
David Denisov. 2027. Client-Side ANN on the Web: A Wikipedia Search Demonstration. In *Companion Proceedings of the ACM Web Conference 2027 (WWW '27)*. ACM, New York, NY, USA, 4 pages.

1 Introduction

Given a dataset of vectors and a query vector, k-nearest neighbour (kNN) retrieval finds the k dataset vectors closest to the query under a chosen distance metric (Euclidean distance in our demo). Approximate nearest neighbour (ANN) search is used to speed up this task on large collections, at the cost of potentially missing some of the true nearest neighbours. In retrieval-augmented generation (RAG), ANN search is used to retrieve passages that a language model uses as context to generate an answer [6].

Conventional ANN systems keep the index on the machine that runs the search, in memory or on disk [1, 10]. On the Web, this typically means running search on a server or downloading the full index to the browser, which would yield prohibitively large data transfer for our corpus. The FP16 vectors for our 35.2 million Wikipedia passages alone occupy about 27 GB before compression.

Our method uses an inverted file (IVF) index built with Faiss [2]. The client fetches selected clusters and ranks their vectors. The demo uses free static CDN hosting, and all query-time computation runs on the client's machine.

1.1 Related work

This demo builds on an earlier public image-search website by the author, omitted here due to its content. The subsequent ANNPack 0.1.x also fetches selected vector clusters over the Web.

SPANN [1] demonstrates IVF search over vectors stored on a local SSD. Our demo applies the same idea to cluster files hosted on a CDN, allowing storage beyond a single machine. Tiptoe [4, Section 4] uses IVF-style cluster selection and computes similarity scores under encryption on a dedicated server.

DiskANN [10] computes ANN by traversing a graph (stored on SSD). MeMemo [11] and WebANNS [7] keep graph indexes in browser RAM and IndexedDB; WebANNS evaluates up to 480,000 passages. Importing our remote corpus into such local indexes would require a large initial data transfer; we avoid that import by fetching only selected clusters.

More broadly, Pagefind (keyword search) and sql.js-httpvfs (SQL) also process selectively downloaded static data in the browser.

2 Demonstration scenario

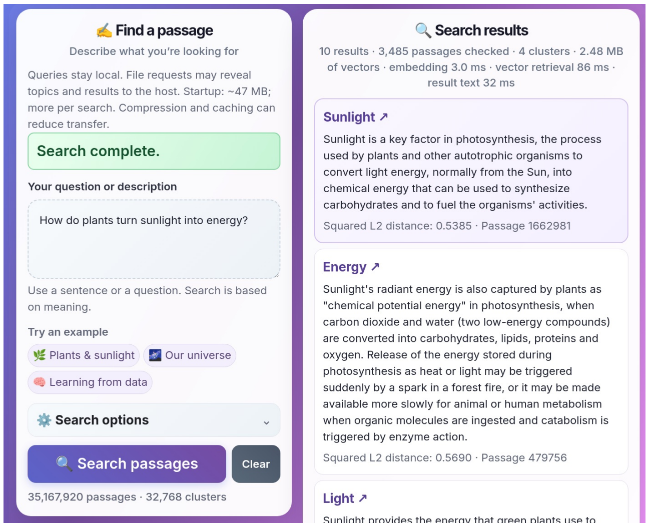
Both public pages use Maloyan's dataset of 35,167,920 passages from a December 2022 snapshot of English Wikipedia [8]. Wikipedia gives visitors a broad choice of topics. A full index download would be prohibitively large. Our method makes browser search practical at this scale with only free static CDN hosting.

Wikipedia search. Visitors can enter any text or questions of their choice and receive ranked passages linked to Wikipedia (Figure 1). Defaults are 32k clusters, SQ6 centroids, four probes, and ten results. Controls offer three cluster counts, six centroid formats, and 1–100 probes. The page reports embedding, centroid-loading, retrieval and text-loading times, and vector downloads.

Clustering lab. The lab (Figure 2) offers noise presets and checks for cluster count, probes and centroid format. It reports top-1 agreement with exhaustive Faiss search. Visitors select 1–10,000 perturbed-passage queries per configuration (default 10). Short runs uniformly sample a shared subset. At the poster, visitors can run short checks and discuss the plots.

Public access. The presenter will preload all required resources on a validated laptop. A QR code and URL on the poster will give visitors access to the demo. For incompatible browsers, saved screenshots and retained plots support a narrated walkthrough. The author commits to keeping the demo URL available for at least three years after the conference.

WWW '27, Dublin, Ireland
2027.



✍️ Find a passage

Describe what you’re looking for

Queries stay local. File requests may reveal topics and results to the host. Startup: ~47 MB; more per search. Compression and caching can reduce transfer.

Search complete.

Your question or description

How do plants turn sunlight into energy?

Use a sentence or a question. Search is based on meaning.

Try an example

🌿 Plants & sunlight
🌌 Our universe
🧠 Learning from data
⚙️ Search options
🔍 Search passages
Clear

35,167,920 passages · 32,768 clusters

🔍 Search results

10 results · 3,485 passages checked · 4 clusters · 2.48 MB of vectors · embedding 3.0 ms · vector retrieval 86 ms · result text 32 ms

Sunlight ↗

Sunlight is a key factor in photosynthesis, the process used by plants and other autotrophic organisms to convert light energy, normally from the Sun, into chemical energy that can be used to synthesize carbohydrates and to fuel the organisms' activities.

Squared L2 distance: 0.5385 · Passage 1662981

Energy ↗

Sunlight's radiant energy is also captured by plants as "chemical potential energy" in photosynthesis, when carbon dioxide and water (two low-energy compounds) are converted into carbohydrates, lipids, proteins and oxygen. Release of the energy stored during photosynthesis as heat or light may be triggered suddenly by a spark in a forest fire, or it may be made available more slowly for animal or human metabolism when organic molecules are ingested and catabolism is triggered by enzyme action.

Squared L2 distance: 0.5690 · Passage 479756

Light ↗

Sunlight provides the energy that green plants use to

Figure 1: Search with privacy/download notices and passages. Capture timings are illustrative; Section 4.4 estimates costs from component means. Text enlarged for clarity.

3 Method outline

Figure 3 traces local computation and the static files downloaded at startup and during search. The probe count sets how many clusters the browser fetches and searches.

We use the Wikipedia passages and precomputed embeddings in Maloyan’s dataset [8]. The `all-MiniLM-L6-v2` encoder [9] produces these 384-dimensional embeddings. We chose MiniLM to balance semantic search quality, model size and inference cost. Its Xenova ONNX conversion [14] lets Transformers.js [3] embed queries locally in the same vector space as the stored passages. The browser uses the q8 model and truncates inputs to 256 tokens.

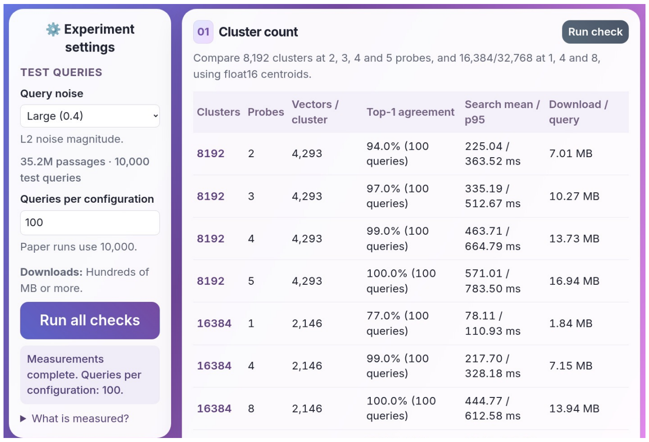
Index preparation. We train C centroids with Faiss k-means [2], using 20 iterations on about 2.1 million uniformly sampled passages. We assign each passage to its nearest centroid and store its ID and FP16 vector in that centroid’s gzipped file. Centroids are published in FP16 and compressed formats. Passage vectors remain FP16, occupying about 27 GB per index layout before compression.

Result text. We group text into Brotli files of 500 consecutive passage IDs, except for the final file, to balance privacy and download size. Larger batches make text requests less specific by covering more possible result IDs, at the cost of more unused text per download. These batches are shared across cluster counts.

Static hosting. On the Cloudflare Pages Free plan, the 70,336 text files use 71 branch deployments. Vectors and centroids use 59 more in a second project. Manifests map files to URLs. Our 1,000-file upload batches are a packaging choice, below the provider’s 20,000-file ceiling.

4 Supporting evaluation

We measure retrieval accuracy against exhaustive search. We time text retrieval separately in groups of ten passages.



⚙️ Experiment settings

TEST QUERIES

Query noise: Large (0.4)

L2 noise magnitude.

35.2M passages · 10,000 test queries

Queries per configuration: 100

Paper runs use 10,000.

Downloads: Hundreds of MB or more.

Run all checks

Measurements complete. Queries per configuration: 100.

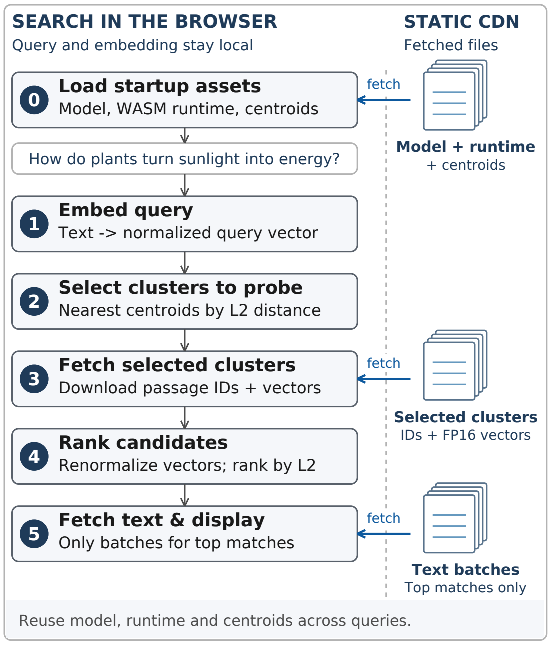
▶ What is measured?
Run check

01 Cluster count

Compare 8,192 clusters at 2, 3, 4 and 5 probes, and 16,384/32,768 at 1, 4 and 8, using float16 centroids.

Clusters	Probes	Vectors / cluster	Top-1 agreement	Search mean / p95	Download / query
8192	2	4,293	94.0% (100 queries)	225.04 / 363.52 ms	7.01 MB
8192	3	4,293	97.0% (100 queries)	335.19 / 512.67 ms	10.27 MB
8192	4	4,293	99.0% (100 queries)	463.71 / 664.79 ms	13.73 MB
8192	5	4,293	100.0% (100 queries)	571.01 / 783.50 ms	16.94 MB
16384	1	2,146	77.0% (100 queries)	78.11 / 110.93 ms	1.84 MB
16384	4	2,146	99.0% (100 queries)	217.70 / 328.18 ms	7.15 MB
16384	8	2,146	100.0% (100 queries)	444.77 / 612.58 ms	13.94 MB

Figure 2: Lab cluster-count check with 100 queries per configuration. Text enlarged for clarity.



SEARCH IN THE BROWSER
Query and embedding stay local
STATIC CDN
Fetched files
0 Load startup assets
Model, WASM runtime, centroids
fetch
Model + runtime + centroids
How do plants turn sunlight into energy?
1 Embed query
Text -> normalized query vector
2 Select clusters to probe
Nearest centroids by L2 distance
3 Fetch selected clusters
Download passage IDs + vectors
fetch
Selected clusters
IDs + FP16 vectors
4 Rank candidates
Renormalize vectors; rank by L2
5 Fetch text & display
Only batches for top matches
fetch
Text batches
Top matches only
Reuse model, runtime and centroids across queries.

Figure 3: Local query embedding and ranking, with model, runtime and centroid downloads at startup and selective vector and text downloads per query.

Query setup. Configurations share 10,000 uniformly sampled distinct source passages. For each sampled passage, we draw a Gaussian vector $v \in \mathbb{R}^{384}$, rescale it to noise magnitude $\|v\|_2 = \alpha$, add it to the passage’s unit embedding, and renormalise to obtain the query. Routing tests use $\alpha = 0.4$. The lab also offers 0.05 and 0.15.

Noise scale. At $\alpha = 0.4$, mean L2 query displacement across 1,000 uniformly sampled passages was 55.5% of the mean exact nearest-neighbour distance (excluding self-matches). Of the three scales tested, probe count affected top-1 agreement most at 0.4.

Metric. **Top-1 agreement** is the fraction of queries whose first browser result has the same passage ID as the top result from an exhaustive Faiss L2 scan over the same renormalised FP16 passage vectors. For the 10,000 high-noise queries, 32k/SQ6 and four probes agree on 98.89%, with a 95% Wilson interval of 98.67% to 99.08% [13].

Test environment. Experiments use Ubuntu 22.04 on an Intel Core i5-12400F PC with 32 GB RAM and a GTX 1660 SUPER GPU. GPU Faiss prepared the indexes. Browser experiments use Chromium 155; inference uses Transformers.js 4.3.1 with CPU-only WebAssembly and four threads. Separate FAST.com checks reported about 300 Mbit/s download; CDN throughput was uncontrolled.

Embedding compatibility. Re-embedding 1,000 uniformly sampled passages with the q8 browser encoder yields mean and 5th-percentile cosine similarities of 0.9924 and 0.9892 to the stored vectors. These values show that the browser embeddings remain compatible despite small differences.

Configurations. FP16 probe sweeps use 2–5 probes at 8k and 2, 3, 4, 6, 8, 10 at 16k/32k. At 32k and four probes, we compare FP16 centroids with versions compressed to SQ8/SQ6/SQ4, symmetric INT8 [2], and PQ4 with 4-bit codes per eight-component block [5].

Hosting and timing. Vector and text files are prefetched to prime CDN caches. CDN residency and connections remain uncontrolled. Timed routing searches return one passage, fetch up to ten clusters in parallel and bypass browser HTTP caches. Untimed accuracy checks allow cached clusters. Table 1 reports agreement, timing and payload on the same timed queries.

Retrieval time includes routing, vector download, decompression, decoding and ranking; Sections 4.3–4.5 report text retrieval, total query costs and startup. Centroids load once per configuration. Download and decode benchmarks use 1,000 trials per file. Decode time includes gzip decompression. Tables give mean and 95th-percentile (p95) times, and gzipped vector payload in decimal MB (10^6 bytes). Configuration order was randomised; the original seed and supplement order are recorded.

4.1 Vector retrieval

With FP16 centroids, four probes agree with exact search on at least 98.96% of queries at all three cluster counts (Table 1). More clusters reduce vector downloads and latency but increase centroid loading.

Table 1: FP16 four-probe retrieval. Top-1 agreement, retrieval time, and vector payload per query. 10,000 **queries per setting.**

			Time (ms)	Payload (MB)
Clusters	Probes	Agree (%)	Mean / p95	Mean / p95
8192	4	99.29	462 / 662	13.65 / 18.38
16384	4	99.02	215 / 326	7.11 / 9.92
32768	4	98.96	135 / 182	3.98 / 5.85

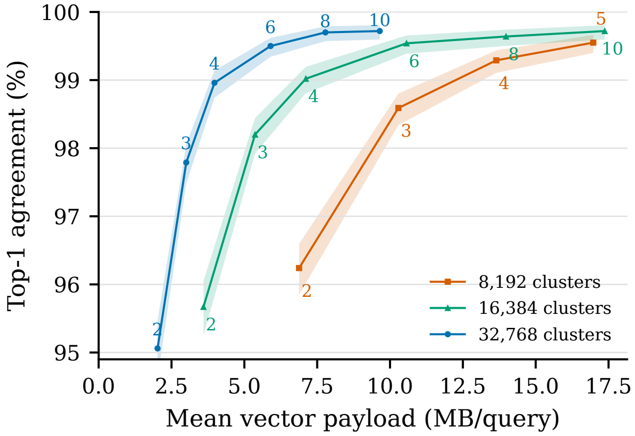


Figure 4: Top-1 agreement versus vector payload (FP16, 10,000 **queries/setting). Shading shows pointwise** 95% **Wilson intervals. Labels mark probes.**

Figure 4 reports gzipped vector payload, excluding startup and text. Payload size is independent of network conditions, which can vary during runs. Observed retrieval time scaled approximately linearly with payload across probe settings.

4.2 Centroid loading costs

At 32k/four probes (Table 2), **SQ6 cuts centroid bytes by** 61.4%, from 23.24 to 8.98 MB. Mean loading falls from 929 ms (FP16) to 324 ms (SQ6), with minor effect on top-1 agreement.

Table 2: Centroid formats at 32k clusters and four probes. Top-1 agreement on 10,000 **queries, file size, and loading mean / p95 over 1,000 trials per file.**

Format	Agree (%)	Size (MB)	Fetch (ms)	Decode (ms)
FP16	98.96	23.24	675 / 692	254 / 260
SQ8	98.99	11.23	300 / 339	113 / 115
SQ6	98.89	8.98	224 / 261	100 / 102
SQ4	98.23	4.86	83 / 102	88 / 91
INT8	98.94	10.99	290 / 342	103 / 106
PQ4	68.73	0.81	23 / 33	30 / 31

FP16, SQ8, SQ6, and INT8 differ by at most 0.10 percentage points. SQ4 and PQ4 reduce agreement at the tested code sizes.

4.3 Text retrieval

We measure 1,000 groups of ten passages, using 10,000 uniformly sampled distinct passage IDs. Text retrieval takes 34.9/54.9 ms (mean/p95), covering parallel batch downloads, Brotli decompression and text decoding. We prefetch files and bypass browser and decoded-text caches. DOM rendering is excluded.

4.4 Total query cost

At 32k/SQ6 and four probes, vector retrieval averages 122.4 ms and ten-passage text loading 34.9 ms. Embedding averages 9.1, 16.9 and 33.8 ms for 64, 128 and 256 tokens, giving estimated totals of 166.4, 174.1 and 191.1 ms, respectively. The estimates sum the means of the three sequential stages from separate runs, excluding startup, rendering, messaging and extra ranking.

4.5 Client setup

With the page open, startup takes 2.46/2.53 s (mean/p95) across 100 fresh Chromium contexts. Startup assets total about 47 MB before HTTP compression. Separate tests measured the main downloads (size, mean time) for model weights (23.0 MB, 1.44 s), WebAssembly (14.3 MB, 324 ms) and 32k/SQ6 centroids (9.0 MB, 224 ms). Library import, model initialization and centroid decoding averaged 371, 642 and 100 ms. Some loads and components run in parallel, so startup takes less than the sum of the separate timings.

4.6 Trade-offs and limitations

SQ8/PQ passage codes with optional rescoring could reduce vector transfer, but their ranking quality is untested.

The timings cover controlled workloads and one live question. Latency across diverse real questions, on mobile devices and on throttled networks remains unmeasured.

Privacy threat model. The open-source client processes query text and embeddings locally. Model and runtime assets load from Hugging Face and jsDelivr. A host, CDN or network observer may link IP addresses and timing to the IDs of requested clusters and text batches. With known centroids, the requested clusters immediately define the region of query embeddings for which those centroids are the nearest. This follows from rearranging the distance comparisons into linear inequalities, which can be constructed in under a second for 32k clusters and four probes. Given the requested clusters and batches, it may be possible to reconstruct the displayed result set or part of it, though how much can be recovered remains an open question. The same unordered file IDs can be consistent with different result sets. Co-location would avoid result-specific text requests but require one text layout per count, while selected clusters remain visible. PIR, as in Tiptoe [4], requires a different serving protocol.

5 Conclusion

SPANN [1] showed that IVF search can use a local SSD. Our demo extends IVF-based ANN search beyond a single machine by serving selected clusters from a static CDN for browser search over 35.2 million passages. The evaluation shows the trade-offs among agreement with exact search, latency, and bandwidth. We hope this approach will be useful for retrieval in RAG systems and for servers that can fetch clusters over high-speed networks, although remote reads can have higher latency than local SSD reads.

6 Ethical use of data

We downloaded Maloyan's precomputed embeddings of 35,167,920 English Wikipedia passages (wikipedia-22-12-en) [8]. They were generated by applying the MiniLM encoder [9] to these passages. The encoder and its Xenova ONNX conversion [14] for browser queries are Apache-2.0 licensed. Result links credit Wikipedia contributors through source articles [12]. The supplied client and text exporters include a CC BY-SA 3.0 notice and `DATA-NOTICE.txt`, recording attribution and extraction/packaging changes. The embedding dataset specifies no separate licence. Archived passages may be outdated or biased, and linked articles may change or disappear. No human participants were involved.

References

[1] Qi Chen, Bing Zhao, Haidong Wang, Mingqin Li, Chuanjie Liu, Zengzhong Li, Mao Yang, and Jingdong Wang. 2021. SPANN: Highly-efficient Billion-scale Approximate Nearest Neighborhood Search. In *Advances in Neural Information Processing Systems*, M. Ranzato, A. Beygelzimer, Y. Dauphin, P.S. Liang, and J. Wortman Vaughan (Eds.), Vol. 34. Curran Associates, Inc., 5199–5212. https://proceedings.neurips.cc/paper_files/paper/2021/file/299dc35e747eb77177d9cea10a802da2-Paper.pdf

[2] Matthijs Douze, Alexandr Guzhva, Chengqi Deng, Jeff Johnson, Gergely Szilvasy, Pierre-Emmanuel Mazaré, Maria Lomeli, Lucas Hosseini, and Hervé Jégou. 2026. The Faiss Library. *IEEE Transactions on Big Data* 12, 2 (2026), 346–361. doi:10.1109/TBDATA.2025.3618474

[3] Hugging Face. 2024. Transformers.js: State-of-the-art Machine Learning for the Web. GitHub repository. https://github.com/huggingface/transformers.js Accessed October 24, 2026.

[4] Alexandra Henzinger, Emma Dauterman, Henry Corrigan-Gibbs, and Nickolai Zeldovich. 2023. Private Web Search with Tiptoe. In *Proceedings of the 29th Symposium on Operating Systems Principles (SOSP '23)*. ACM, 396–416. doi:10.1145/3600006.3613134

[5] Hervé Jégou, Matthijs Douze, and Cordelia Schmid. 2011. Product Quantization for Nearest Neighbor Search. *IEEE Transactions on Pattern Analysis and Machine Intelligence* 33, 1 (2011), 117–128. doi:10.1109/TPAMI.2010.57

[6] Patrick Lewis, Ethan Perez, Aleksandra Piktus, Fabio Petroni, Vladimir Karpukhin, Naman Goyal, Heinrich Küttler, Mike Lewis, Wen-tau Yih, Tim Rocktäschel, Sebastian Riedel, and Douwe Kiela. 2020. Retrieval-Augmented Generation for Knowledge-Intensive NLP Tasks. In *Advances in Neural Information Processing Systems*, Vol. 33. Curran Associates, Inc., Red Hook, NY, USA, 9459–9474. https://proceedings.neurips.cc/paper/2020/hash/6b493230-Abstract.html

[7] Mugeng Liu, Siqi Zhong, Qi Yang, Yudong Han, Xuanzhe Liu, and Yun Ma. 2025. WebANNS: Fast and Efficient Approximate Nearest Neighbor Search in Web Browsers. In *Proceedings of the 48th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '25)*. Association for Computing Machinery, New York, NY, USA, 2483–2492. doi:10.1145/3726302.3730115

[8] Narek Maloyan. 2023. Wikipedia-22-12-en embeddings with all-MiniLM-L6-v2. Hugging Face dataset. https://huggingface.co/datasets/maloyan/wikipedia-22-12-en-embeddings-all-MiniLM-L6-v2 Accessed October 24, 2026.

[9] sentence-transformers. n.d. all-MiniLM-L6-v2. Hugging Face model card. https://huggingface.co/sentence-transformers/all-MiniLM-L6-v2 Accessed October 30, 2026.

[10] Suhas Jayaram Subramanya, Devvrit, Rohan Kadekodi, Ravishankar Krishnaswamy, and Harsha Vardhan Simhadri. 2019. DiskANN: fast accurate billion-point nearest neighbor search on a single node. In *Advances in Neural Information Processing Systems (NeurIPS)*.

[11] Zijie J. Wang and Duen Horng Chau. 2024. MeMemo: On-device Retrieval Augmentation for Private and Personalized Text Generation. In *Proceedings of the 47th International ACM SIGIR Conference on Research and Development in Information Retrieval (SIGIR '24)*. Association for Computing Machinery, New York, NY, USA, 2765–2770. doi:10.1145/3626772.3657662

[12] Wikipedia contributors. n.d. Wikipedia:Copyrights. https://en.wikipedia.org/wiki/Wikipedia:Copyrights Accessed October 30, 2026.

[13] Edwin B. Wilson. 1927. Probable Inference, the Law of Succession, and Statistical Inference. *J. Amer. Statist. Assoc.* 22, 158 (June 1927), 209–212. doi:10.1080/01621459.1927.10502953

[14] Xenova. n.d. Xenova/all-MiniLM-L6-v2. Hugging Face model card. https://huggingface.co/Xenova/all-MiniLM-L6-v2 Accessed October 30, 2026.